

1.4.14. 4-sum. Tìm thuật toán cho bài tổng-4

- dùng 2 vòng lặp for để tạo ra $n(n-1)/2$ cặp số
- sau đó dùng quick sort hoặc merge sort để sắp xếp từ bé đến lớn
- dùng lệnh if để kiểm tra nếu tổng của 2 cặp = 0 mà lại có số trùng ở 2 cặp
- dùng kĩ thuật 2 pointer để ghép từng cặp với nhau nếu tổng = 0 thì count +1 còn ko thì giữ nguyên
- độ phức tạp là $n^2 \log n$

1.4.15. *Faster 3-sum*. Để khởi động, hãy tìm thuật toán và cài TwoSumFaster sử dụng một thuật toán tuyến tính để đếm số cặp có tổng bằng 0 đối với mảng đã được sắp xếp (thay vì dùng tìm kiếm nhị phân như trong thuật toán $N \log N$). Sau đó áp dụng ý tưởng này để phát triển một thuật toán bậc hai cho bài tổng-3

- làm y hệt bài trên

1.4.16. *Closest pair*. Viết một chương trình nhận input là một mảng $a[]$ gồm N giá trị double, tìm một cặp gần nhất (closest pair) - là hai giá trị có hiệu không vượt quá hiệu của cặp số bất kì khác trong mảng. Hiệu ở đây tính bằng giá trị tuyệt đối. Thuật toán cần có thời gian chạy $N \log N$ trong trường hợp tồi nhất.

- dùng merge sort
- // Giả sử mảng $a[]$ đã có dữ liệu
- Merge.sort(a);
-
- double minDiff = Double.MAX_VALUE; // Đặt giá trị nhỏ nhất tạm thời là vô cực
- double num1 = 0, num2 = 0;
-
- // so sánh các cặp "hàng xóm"
- for (int i = 1; i < a.length; i++) {
- double diff = a[i] - a[i-1]; if (diff < minDiff) {
- minDiff = diff;
- num1 = a[i-1];
- num2 = a[i];
- }
- }
-
- System.out.println("Cặp gần nhất: " + num1 + " và " + num2);
-

1.4.17. *Farthest pair*. Viết một chương trình nhận input là một mảng $a[]$ gồm N giá trị double, tìm một cặp xa nhất (Farthest pair) - là hai giá trị có hiệu không nhỏ hơn hiệu của cặp số bất kì khác trong mảng. Hiệu ở đây tính bằng giá trị tuyệt đối. Thuật toán cần có thời gian chạy tuyến tính trong trường hợp tồi nhất.

- // Giả sử mảng $a[]$ có ít nhất 2 phần tử
- double min = a[0];
- double max = a[0];

```
-  
- // Quét đúng 1 vòng duy nhất (Tổn O(N))  
- for (int i = 1; i < a.length; i++) {  
-     if (a[i] < min) {  
-         min = a[i]; // Ghi nhận đáy mới  
-     }  
-     if (a[i] > max) {  
-         max = a[i]; // Ghi nhận đỉnh mới  
-     }  
- }  
-  
- System.out.println("Cặp xa nhất là: " + min + " và " + max);  
- System.out.println("Khoảng cách xa nhất là: " + (max - min));  
-
```